

IA Agent

**Kordoghli Salim
Mdalla Talel**

Table de Contenu

1. Informations générales sur l'Agent
 - 1.1. Comment il fonctionne
 - 1.2. Pourquoi Gemini 2.0 Flash Lite
2. Présentation de la base des tests
3. Test sur 10+ fonctions
 - 3.1. Présentation du fichier de test et ses propriétés
 - 3.2. Exécution du test et comparaison des résultats
4. Test sur 100+ fonctions
 - 4.1. Présentation du fichier de test et ses propriétés
 - 4.2. Exécution du test et comparaison des résultats

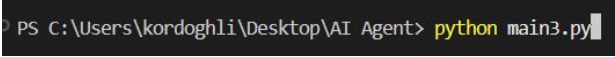
Informations sur l'Agent

Cet agent est en réalité composé de deux modules indépendants mais complémentaires :

- l'un est spécifiquement dédié à l'analyse du code source, tandis que l'autre se concentre sur les fichiers de tests unitaires.
- Cette séparation fonctionnelle résulte d'une série de tests approfondis, qui ont mis en évidence la robustesse et la clarté de cette architecture modulaire, en comparaison avec une approche centralisée reposant sur un agent unique chargé de l'ensemble des tâches.

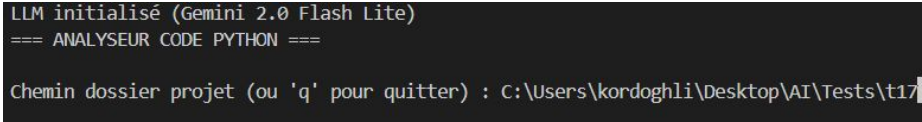
Comment il fonctionne

1. Ouvrir le fichier Python dans l'éditeur de votre choix (ex. : VS Code, PyCharm ...).
2. Exécuter le script via le terminal en utilisant la commande suivante : `python nomdelagent.py`

Exemple : 

3. Une interface en ligne de commande s'affiche et vous invite à saisir le chemin du dossier à analyser, ou à taper `q` pour quitter.
4. Entrer le chemin du dossier contenant le code source et/ou les tests unitaires à évaluer.

Exemple :



```
LLM initialisé (Gemini 2.0 Flash Lite)
=== ANALYSEUR CODE PYTHON ===
Chemin dossier projet (ou 'q' pour quitter) : C:\Users\kordoghli\Desktop\AI\Tests\t17
```

5. Une analyse complète est alors lancée à l'aide de Gemini 2.0 Flash lite. Les résultats détaillés s'affichent dans le terminal.
Par ailleurs, un **rapport synthétique contenant uniquement les métriques principales** est automatiquement enregistré dans un dossier prédéfini.
Ce fichier peut être utilisé ultérieurement pour divers besoins tels que l'intégration dans un tableau de bord, la comparaison de projets (**benchmarking**) ou l'entraînement de modèles prédictifs.

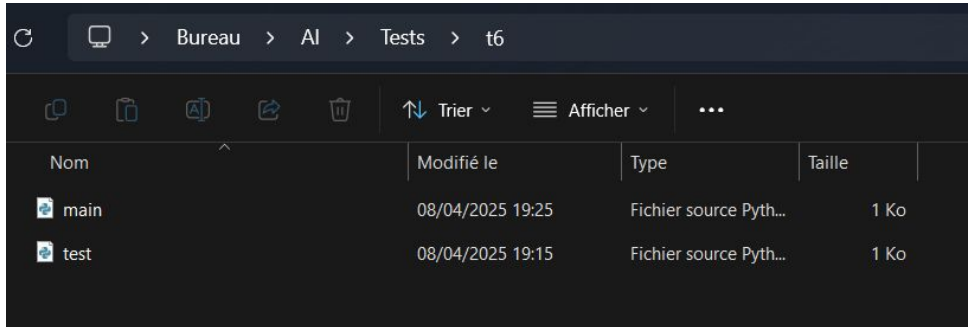
Pourquoi Gemini 2.0 Flash Lite

Comme évoqué dans le tableau suivant, ces sont les limitations de version gratuite de Gemini, Puisque l'agent ne nécessite pas d'énormes ressources pour fonctionner fiablement, nous devons choisir un modèle qui assure le plus de crédit avec un bon niveau d'analyse, ce qui est le cas de Gemini 2.0 Flash Lite

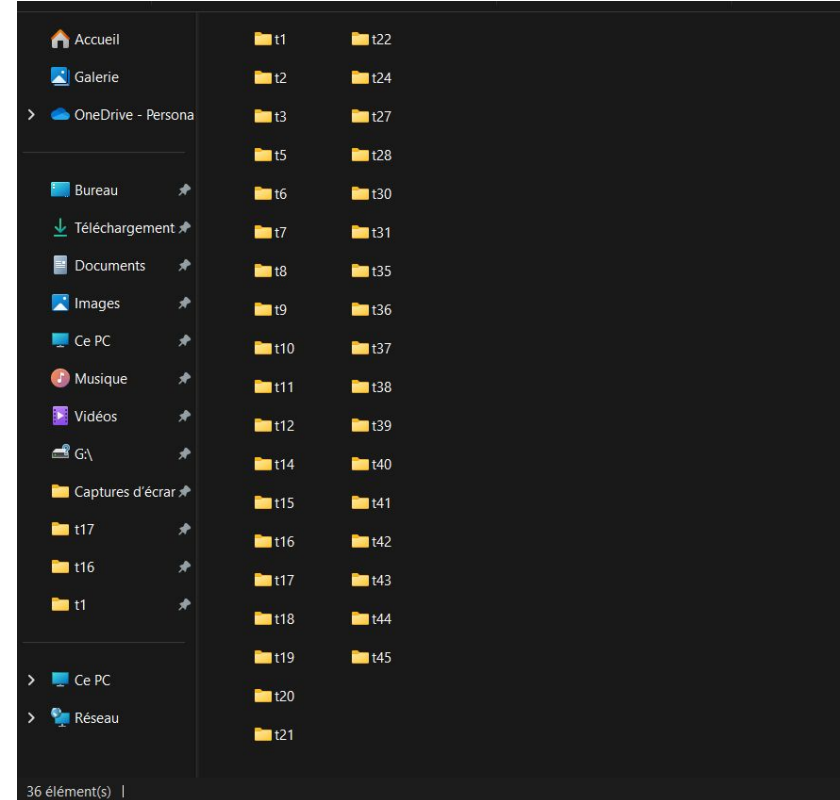
Version gratuite Niveau 1 Niveau 2 Catégorie 3			
Modèle	RPM	TPM	RPD
Gemini 2.5 Flash Preview 05-20	10	250 000	500
Gemini 2.5 Pro Preview 05-06	--	--	--
Gemini 2.5 Pro Experimental 03-25	5	250 000 TPM 1 000 000 TPD	25
Gemini 2.0 Flash	15	1 000 000	1 500
Génération d'images d'aperçu Gemini 2.0 Flash	10	200 000	100
Gemini 2.0 Flash Experimental	10	1 000 000	1 000
Gemini 2.0 Flash-Lite	30	1 000 000	1 500
Gemini 1.5 Flash	15	250 000	500
Gemini 1.5 Flash-8B	15	250 000	500

La base des tests

La base de tests utilisé contient 36 fichiers de tests , La structure de chaque dossier est ci-dessous , un fichier contenant le code à analyser et l'autre le test unitaire qui correspond au code (La structure change d'un dossier à un autre pour subvenir à tous les cas possibles)



Nom	Modifié le	Type	Taille
main	08/04/2025 19:25	Fichier source Python	1 Ko
test	08/04/2025 19:15	Fichier source Python	1 Ko



Les cas interprétés lors du choix des tests à appliquer

- Codes bien documentés, bien commentés avec des tests unitaires claires et pertinents (t30 comme exemple)
- Codes bien documentés , bien commentés avec des tests unitaires erronés (t2 comme exemple)
- Codes bien documentés, mal commentés avec des tests unitaires erronés ou n'existent pas (t1 comme exemple)
- Codes mal documentés , mal commentés et sans tests unitaires (t24)
- Fichier de Tests unitaires claires uniquement sans code source (t17 par exemple)
- Fichier de type qui n'est même pas python (t11 comme exemple)
- Codes sources de plus de 15 fonctions , allant jusqu'à 112 fonctions (t24)

Test sur 10+ fonctions

On exécute notre analyse sur "t16" , Un code qui contient 10 fonctions diverses

Les propriétés de ce test :

- 1 seule fonction est documentée , et sa documentation est erronée (Add_book) : La fonction ajoute un livre à une bibliothèque et la documentation décrit une fonction de multiplication
- 1 seule fonction commenté , et le commentaire est bien précis : Update_year
- Le fichier de test unitaire existe
- Dans le fichier de test, la deuxième assertion test_remove_book est erronée , donc l'Agent doit détecter exactement cette anomalie et attribuer 0 à tout le test unitaire

Résultats du test

Métriques Finales

Métrique	Valeur
Files Exist	1
Files Valid	1
Has Tests	1
Tests Valid	0
Fully Documented	0
All Functions Commented	0
Docstrings Accurate	0.00
Comments Accurate	0.10

Sauvegarde rapport : C:\Users\kordoghli\Desktop\AI\Reports\report_t16.json

Rapport sauvegardé : C:\Users\kordoghli\Desktop\AI\Reports\report_t16.json

```
Métriques retournées : {  
  "files_exist": 1,  
  "files_valid": 1,  
  "has_tests": 1,  
  "tests_valid": 0,  
  "fully_documented": 0,  
  "all_functions_commented": 0,  
  "docstrings_accurate": 0.0,  
  "comments_accurate": 0.1  
}
```

Qualité Code

Fonctions : 10
Syntaxe Valide : Oui
Fonctions Sans Docstring : 9/10
Fonctions Sans Commentaire : 9/10
Docstrings Précises : 0.00
Commentaires Précis : 0.10

Fonctions Non Commentées :

- add_book
- remove_book
- update_author
- borrow_book
- return_book
- is_book_available
- get_book_info
- list_available_books
- count_books_by_author

Fonctions Non Documentées :

- remove_book
- update_author
- update_year
- borrow_book
- return_book
- is_book_available
- get_book_info
- list_available_books
- count_books_by_author

Section des docstrings

****Problèmes Docstrings :**b>**

- `add_book` (Docstring : "Calcule le produit de deux nombres.") :
Raison : La docstring est incorrecte car elle décrit une multiplication alors que la fonction ajoute un livre à une bibliothèque. Il n'y a aucune correspondance entre la description et le code.
- La docstring décrit une opération de multiplication, alors que la fonction ajoute un livre à une bibliothèque (dictionnaire).
- La docstring est complètement fausse car elle ne décrit pas du tout le comportement de la fonction.

L'Agent ici a bien interprété l'erreur logique qui existe dans la documentation : La contradiction majeure qui existe entre l'utilité de la fonction (Ajouter un livre à une bibliothèque) et la documentation rédigée (Calcule le produit de deux nombres) et a bien expliqué la cause de cette contradiction

Section des commentaires

- update_year (Commentaire : "Met à jour l'année") :

Raison : Le commentaire décrit correctement l'objectif principal de la fonction : mettre à jour l'année d'un livre dans une bibliothèque. Il est précis et ne contient pas d'erreurs majeures.

L'Agent ici a encore bien interprété le commentaire, l'a contribué comme valide , a expliqué la cause de sa décision brièvement et sans contraintes

Section des tests unitaires

```
=====
RÉSUMÉ DE L'ANALYSE
=====
Score: 0 (Tests non valides)
Méthodes de test erronées: test_remove_book
```

L'Agent ici a bien détecté les 5 méthodes de tests qui existent et la méthode erronée : `test_remove_book` et a attribué 0 à la validité du test unitaire entier

```
Étape: Analyse statique de la structure du fichier...
Fichier parsé avec succès
Recherche des classes de test...
- Méthode trouvée: test_add_book
- Méthode trouvée: test_remove_book
- Méthode trouvée: test_borrow_book
- Méthode trouvée: test_count_books_by_author
- Méthode trouvée: test_is_book_available
Classe de test trouvée: TestLibrary avec 5 méthode(s)

Étape: Vérification des assertions...
test_add_book: Assertions trouvées
test_remove_book: Assertions trouvées
test_borrow_book: Assertions trouvées
test_count_books_by_author: Assertions trouvées
test_is_book_available: Assertions trouvées
Des assertions ont été trouvées dans certains tests
```

Test sur 100 + fonctions

On exécute notre analyse sur “t24” , Un code qui contient 112 fonctions diverses

Les propriétés de ce test :

- Tous les fonctions (112) sont documentés
- 21 fonctions sont commentés
- Aucun fichier de test unitaire n'est présent
- Parmi les 112 fonctions documentés, Il existe 7 fonctions qui ne sont pas bien documentés
- Parmi les 21 fonctions commentés , il existe 10 commentaires qui ne sont bien commentés : Soit par contradiction ou le commentaire est assez vague qu'il ne doit pas être accepté

Résultats de l'analyse

```
Score docstrings_accurate : 0.94 (105/112)
```

```
Score comments_accurate : 0.10 (11/112)
```

```
=== RAPPORT FINAL ANALYSE CODE ===
```

```
### Structure Fichiers
```

```
Implémentation : C:\Users\kordoghli\Desktop\AI\Tests\t24\main.py
```

```
Fichier Tests : Introuvable
```

```
### Qualité Code
```

```
Fonctions : 112
```

```
Syntaxe Valide : Oui
```

```
Fonctions Sans Docstring : 0/112
```

```
Fonctions Sans Commentaire : 91/112
```

```
Docstrings Précises : 0.94
```

```
Commentaires Précis : 0.10
```

```
**Fonctions Non Commentées :**
```

- fibonacci
- factorielle
- pgcd
- ppcm
- somme_carres
- moyenne_geometrique
- distance_euclidienne
- convertir_base
- inverser_mots

```
### Métriques Finales
```

Métrique	Valeur
Files Exist	1
Files Valid	1
Has Tests	0
Tests Valid	0
Fully Documented	1
All Functions Commented	0
Docstrings Accurate	0.94
Comments Accurate	0.10

```
Sauvegarde rapport : C:\Users\kordoghli\Desktop\AI\Reports\report_t24.json
```

```
Rapport sauvegardé : C:\Users\kordoghli\Desktop\AI\Reports\report_t24.json
```

```
Métriques retournées : {
```

```
  "files_exist": 1,  
  "files_valid": 1,  
  "has_tests": 0,  
  "tests_valid": 0,  
  "fully_documented": 1,  
  "all_functions_commented": 0,  
  "docstrings_accurate": 0.9375,  
  "comments_accurate": 0.09821428571428571
```

```
}
```


Extrait de la section des commentaires

- `est_premier` (Commentaire : "Verifie uniquement jusqu'a la racine carree") :
Raison : Le commentaire décrit avec précision l'optimisation de la fonction qui ne vérifie les diviseurs que jusqu'à la racine carrée du nombre donné. Cela correspond exactement au comportement du code.
- `aire_triangle` (Commentaire : "Semi-perimetre") :
Raison : Le commentaire décrit une étape intermédiaire (le calcul du semi-périmètre) mais pas l'objectif principal de la fonction, qui est de calculer l'aire du triangle. Par conséquent, la précision est inférieure à 65%.
 - Le commentaire décrit le semi-périmètre, pas l'objectif principal de la fonction (calcul de l'aire).
- `fusionner_listes_triees` (Commentaire : "Ajouter les elements restants") :
Raison : Le commentaire décrit correctement une partie du comportement de la fonction : l'ajout des éléments restants des listes après la comparaison des éléments. Bien que le commentaire ne décrive pas l'intégralité du processus de fusion et de tri, il décrit une partie essentielle de la fonction avec précision.
- `formater_numero_telephone` (Commentaire : "Supprime tous les caracteres non numeriques") :
Raison : Le commentaire décrit correctement l'action principale de la fonction, qui est de supprimer les caractères non numériques. Bien qu'il ne mentionne pas le formatage spécifique du numéro de téléphone français, la suppression des caractères non numériques est l'étape cruciale pour ce formatage. Le commentaire est donc précis à plus de 65%.
- `normaliser_texte` (Commentaire : "Supprime les accents") :
Raison : Le commentaire décrit une partie du comportement de la fonction (suppression des accents), mais omet un aspect essentiel : la conversion en minuscules. Par conséquent, la précision est inférieure à 65%.
 - Le commentaire ne mentionne pas la conversion en minuscules, qui fait partie du comportement de la fonction.
- `dernier_jour_mois` (Commentaire : "Aller au premier jour du mois suivant puis reculer d'un jour") :
Raison : Le commentaire décrit fidèlement le comportement de la fonction. Il explique correctement que la fonction calcule le dernier jour du mois en se déplaçant d'abord au premier jour du mois suivant, puis en reculant d'un jour.

Extrait de section des documentations

- `generer_initiales` (Docstring : "Genere les initiales d'un nom complet.") :
Raison : La docstring décrit correctement la fonction. Elle indique que la fonction génère les initiales d'un nom complet, ce qui est le comportement exact de la fonction. La docstring ne contient pas d'informations incorrectes ou contradictoires.
- `masquer_donnees_sensibles` (Docstring : "Masque les donnees sensibles dans un texte.") :
Raison : La docstring est trop générale et ne décrit pas suffisamment le comportement de la fonction. Elle manque de détails importants sur le processus de masquage et les types de données concernées, ce qui rend la description imprécise.
 - La docstring est trop vague. Elle ne précise pas comment les données sensibles sont masquées (remplacement partiel, entier, etc.) ni quels types de données sont concernés.
 - La docstring ne mentionne pas les formats spécifiques de données sensibles qui sont masqués (ex: emails, numéros de téléphone français).
 - La docstring ne précise pas le caractère de masquage utilisé par défaut.
- `generer_token_session` (Docstring : "Genere un token de session aleatoire.") :
Raison : La docstring est trop vague. Elle décrit seulement qu'un token de session est généré, mais ne mentionne pas les aspects clés comme l'aléatoire ou la longueur du token, qui sont des éléments importants de la fonction.
 - La docstring ne spécifie pas que le token est aléatoire.
 - La docstring ne spécifie pas la longueur du token.
- `valider_force_mot_de_passe` (Docstring : "Calcule un score de force pour un mot de passe (0-100).") :
Raison : La docstring décrit correctement la fonction. Elle indique qu'elle calcule un score de force pour un mot de passe entre 0 et 100, ce qui est exact.
- `detecter_injection_sql_simple` (Docstring : "Detecte des tentatives d'injection SQL simples.") :
Raison : La docstring décrit correctement le comportement de la fonction. Elle détecte des tentatives d'injection SQL simples en recherchant des mots-clés suspects dans la requête fournie. La docstring est donc précise.
- `nettoyer_input_utilisateur` (Docstring : "Nettoie un input utilisateur des caracteres potentiellement dangereux.") :
Raison : La docstring décrit correctement l'objectif principal de la fonction : nettoyer l'input utilisateur des caractères dangereux. Bien qu'elle ne détaille pas toutes les étapes (suppression des caractères de contrôle, limitation de la longueur, échappement HTML et suppression des espaces en début et fin), elle capture l'intention générale avec une précision suffisante pour dépasser le seuil de 65%.
- `mesurer_temps_execution` (Docstring : "Mesure le temps d'execution d'une fonction.") :
Raison : La docstring décrit correctement la fonction. Elle indique qu'elle mesure le temps d'exécution d'une fonction, ce qui est le cas. Elle ne donne pas de détails superflus, et la description est donc précise.

Comparaison des tests

L' Agent a indiqué :

- La présence de 112 fonctions
- L'absence de test unitaire
- Présence de tous les docstrings
- Absence de 91 commentaires , par conséquent la présence de 21 commentaires
- **10 Commentaires erronés** : limiter_taux_appels , nettoyer_input_utilisateur, valider_force_mot_de_passe, masquer_donnees_sensibles, nettoyer_html_simple, decompression_rle, calculer_entropie_shannon, valider_isbn, normaliser_texte, aire_triangle
- **7 Documentations erronés** : generer_token_session , masquer_donnees_sensibles, nettoyer_html_simple, decompression_rle, compression_rle, generer_mot_de_passe, valeurs_par_defaut, formater_monnaie

En conclusion, l'Agent a bien identifié les failles présentes dans ce code , donc un score de précision (1) a été attribué lors de ce test